Project 2: 2-Wide Superscalar Tomasulo-Based CPU

Sangeeth Thayaaparan
RIN: 662037988

March 28, 2026

Tools: ModelSim - Intel FPGA Starter Edition 2020.1, SystemVerilog RTL

Abstract

This report describes the design, implementation, and verification of a 2-wide superscalar processor implementing Tomasulo-style out-of-order execution. The CPU features in-order dispatch of up to 2 instructions per cycle into 3 separate reservation stations (for addition, multiplication, and load/store operations), with pipelined functional units and tag-based result forwarding via a Common Data Bus (CDB). An in-order Reorder Buffer (ROB) retires instructions sequentially, maintaining program semantics. The Register Alias Table (RAT) performs dynamic register renaming to eliminate write-after-read and write-after-write hazards, while the dispatch logic enforces read-after-write dependencies. Seven test cases validate single-instruction execution, data dependencies, multiply latency, memory operations, and instruction-level parallelism. Results are verified by comparing final memory writes to expected signatures.

Contents

List of Figures

List of Tables

1 Spec and Rationale

1.1 Instruction Set Architecture (ISA) Subset

The processor supports a minimal subset of RISC-V 32-bit instructions:

- **Immediate (I-type)**: `addi` (add immediate)
- **Register (R-type)**: `add` (addition), `mul` (multiplication)
- **Store (S-type)**: `sw` (store word)
- **Load (I-type variant)**: `lw` (load word)
- **Special**: `nop` (no-op, encoded as `addi x0, x0, 0`)

Out-of-Scope: Branch instructions, jumps, floating-point, CSR operations, exception handling, and cache coherence (single-core only).

1.2 Microarchitecture Parameters

Parameter	**Value**
Dispatch width	2 instructions/cycle
Reorder Buffer (ROB) size	8 entries
Reservation Station depths	ADD: 3, MUL: 3, LS: 4
Functional Unit Latencies:	
ADD (4-stage pipeline)	4 cycles
MUL (6-stage pipeline)	6 cycles
Load/Store execution latency	1 cycle per operation
Register Alias Table entries	32 (RISC-V)
Memory size (unified)	4,096 words (16 KB)

Table 1: Key Microarchitecture Parameters

1.3 Data Path Features and Design Decisions

- **In-order dispatch, out-of-order execution**: Instructions are dispatched in program order but can execute and complete out-of-order, limited only by data dependencies and functional unit availability.
- **Reservation stations per FU type**: Separate RS for add-like, multiply, and load-store operations, enabling independent issue prioritization within each data type.
- **Tag-based result forwarding**: A Common Data Bus (CDB) broadcasts completion as {valid, ROB tag, result}. All RS entries snoop the CDB and capture results matching their pending source tags.
- **In-order ROB retirement**: The ROB commits results sequentially, preserving program order for memory semantics and interrupt handling (if needed).

- **Hazard detection and dispatch control**: The hazard module detects read-after-write (RAW) dependencies, structural conflicts (RS or ROB full), and prevents same-cycle dispatch of FU-type conflicts. It also injects same-cycle slot0-to-slot1 RAW tag forwarding via the RAT output.

1.4 System Architecture

The Tomasulo pipeline is organized as a sequence of 7 stages, each containing specialized modules that perform their designated function independently. Instructions flow left-to-right through the pipeline, with critical feedback paths (CDB, hazard signals) flowing back upstream. Figure **??** illustrates the complete system organization.

Pipeline Flow: Instructions enter through the **Fetch** stage (ifetch module), which maintains a 2-wide instruction queue and outputs the next two instructions per cycle. The **Decode** stage (two parallel tdecode instances) extracts opcode, register indices, immediates, and operation type. The **Rename** stage (rat, Register Alias Table) maps logical registers to physical tags, marking which source operands are available immediately versus pending on a future result. The **Dispatch** stage (hazard detector, dispatchunit, and dispatchpack modules) enforces structural and data dependencies, gates dispatch permission, and packages instructions for broadcast. The **Issue** stage (three reservation stations: rsadd, rsmul, rsls, plus rsarbiter) buffers ready instructions and selects the oldest ready entry per functional unit type. The **Execute** stage (fu_pipe for ADD/MUL, fu_ls for load-store) performs the actual computation or memory operation. The **Writeback** stage (CDB and ROB) broadcasts results and retires instructions in program order.

Common Data Bus (CDB) and Forwarding: The CDB is the critical feedback mechanism in the Tomasulo design. When a functional unit completes an instruction (ADD, MUL, or load), it broadcasts $\{\text{cdbv}, \text{cdbt}, \text{result}\}$ on the CDB. Every reservation station and the ROB snoop this broadcast. If an entry's source operand tag matches cdbt, the entry captures the result in the operand field and clears the pending-tag flag. This tag-matching forwarding allows dependent instructions to issue immediately upon their producer's completion, without stalling or performing classical register-file forwarding. The RAT also snoops the CDB to mark physical registers as ready again.

Hazard Detection and Dispatch Control: Before an instruction dispatches into a reservation station, the hazard module checks for read-after-write (RAW) dependencies (using RAT outputs), structural conflicts (RS or ROB full), and same-FU-type contentions. If a stall condition is detected, the dispatch unit holds the instruction. In the case of a same-cycle slot0-to-slot1 RAW dependency, the hazard module injects slot0's new ROB tag as slot1's source operand, allowing both to dispatch without stalling. This is Tomasulo's key optimization: in-order dispatch without in-order execution.

Memory and Retirement: Load and store operations execute in the LS functional unit (fu_ls) and communicate with the unified 4KB memory. Loads (1-cycle latency) read data and broadcast results on the CDB. Stores write to memory and assert a done signal without broadcasting a result. All memory operations are in-order: they can only issue in program order (enforced by the ROB) and retire sequentially, ensuring memory semantics are preserved. The ROB, with 8 entries, tracks all in-flight instructions. When the oldest entry (ROB slot 0) has its result written back (snooped from CDB or detected when a store completes), the ROB retires that instruction, freeing the entry for a new dispatch.

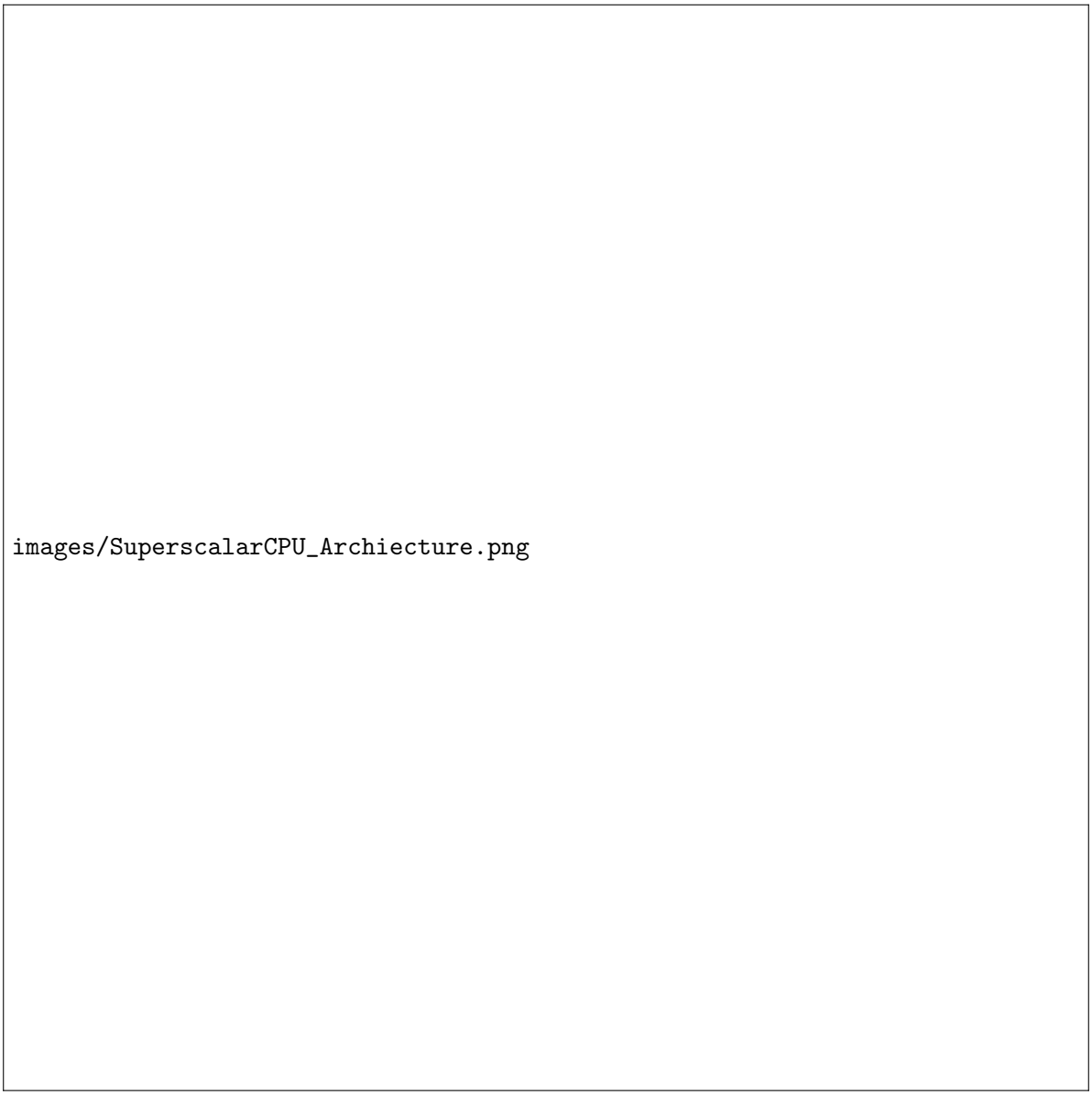

Figure 1: Tomasulo pipeline system architecture showing 7 stages, 13 core modules, interconnecting buses, and CDB feedback loop. Each stage is color-coded and contains the modules responsible for that stage's operations.

2 Microarchitecture and Cycle Behavior

2.1 Top-Level Pipeline Architecture

The Tomasulo pipeline follows a sequence of stages: *fetch*, *decode*, *rename*, *dispatch*, *issue*, *execute*, and *writeback/retire*. Unlike traditional pipelines, issue and execute are decoupled via reservation stations and the Common Data Bus (CDB).

2.1.1 High-Level Organization

1. **Fetch (ifetch)**: 2-wide instruction queue outputs up to 2 instructions per cycle with validity signals
2. **Decode (tdecode x2)**: Each instruction is disassembled into opcode, register indices, immediates, and control signals
3. **Rename (RAT)**: Logical registers are mapped to ROB tags (or values if already committed). Register write suppression for `x0`.
4. **Dispatch (dispatchpack + dispatchunit + hazard)**: Hazard detection gates dispatch; packing bundles instruction metadata into RS-friendly format
5. **Reservation Stations (rsadd, rsmul, rsls)**: Buffers instructions until all operands are ready (either as values or CDB-broadcasted tags)
6. **Issue (rsarbiter)**: Selects oldest ready instruction per FU type; forwards operands to functional units
7. **Execute (fu_pipe or fu_ls)**: Pipelined functional units compute results; broadcast completion to CDB
8. **Writeback/Retire (ROB + CDB)**: Results broadcast on CDB; ROB receives and commits in program order

2.1.2 Data Forwarding via Common Data Bus (CDB)

A critical feature enabling out-of-order execution is the Common Data Bus:

- **Broadcast**: Each cycle, at most one completion broadcasts $\{\text{valid}, \text{tag}, \text{result}\}$
- **Snoop**: All reservation station entries continuously compare their pending source tags against the CDB tag
- **Capture**: When a match is detected, the entry captures the result value and clears the dependency
- **Priority**: If multiple FUs complete in the same cycle, a fixed priority selects which broadcasts (ADD -¿ MUL -¿ LS, implementable as done signal ordering)

The CDB decouples producer and consumer in issue time, allowing consumer instructions to issue immediately after their producer completes, without waiting for writeback to the physical register file (if one existed).

2.1.3 Key Structural Characteristics

- **In-order retirement**: ROB slot 0 always retires if its result has been written; ensures sequential memory/exception semantics
- **Structural hazards**: Dispatch can stall if ROB is full (no free slots) or any target RS is full; prevents over-subscription

- **Data hazards**: Hazard module detects RAW dependencies and suppresses dispatch of dependent instructions in the next cycle (unless tag is injected same-cycle for slot1)
- **No speculation**: No branch prediction, no memory prefetch, no load-store reordering; memory operations issue and complete in program order through ROB

2.2 Module Descriptions

Each module is described following a consistent template: its behavioral role, critical state elements, and key RTL snippets from the implementation.

2.2.1 Module: ifetch (Instruction Fetch Queue)

Behavior: The `ifetch` module is a 2-wide instruction queue that buffers fetched instructions from the unified memory. It outputs up to 2 instructions per cycle along with validity signals. On each cycle where dispatch unit consumes instruction(s), the queue pops the corresponding number of words and refills from memory. The queue never stalls fetch; it stalls dispatch when full (no space for new instructions).

Key signals:

- `pop[1:0]`: dispatch control signal indicating how many instructions to consume (0, 1, or 2)
- `instr0, instr1`: 32-bit instruction outputs (may be valid or invalid)
- `valid0, valid1`: validity flags; when 0, instruction is NOP or should be ignored
- `done`: asserted when all instructions have been fetched and queue is empty on the last pop

Key State:

- `prog[QDEPTH-1:0]`: instruction buffer array (default depth: 1024 words)
- `head`: queue head pointer (word index)
- `proglen`: total program length loaded from file

Implementation:

Listing 1: Instruction Queue Pop and Fetch from Memory

2.2.2 Module: tdecode (Instruction Decoder) [x2]

Behavior: Two instances decode each instruction independently, extracting opcode, register indices, immediates, and control signals. Decodes RISC-V I/R/S-type instruction formats.

Key signals:

- `instr`: 32-bit instruction input
- `op`: 3-bit opcode (OPADD, OPMUL, OPLW, OPSW, OPNOP)
- `kind`: 2-bit FU type (KADD, KMUL, KLS for which reservation station)
- `rs1, rs2, rd`: register indices (5 bits each)
- `imm`: immediate value (sign-extended, 32 bits)
- `valid`: instruction is valid and not NOP
- `regw`: register write enable (0 for stores, jumps, or x0 writes)

Key State: Combinational logic; no state.

Implementation:

```
always_comb begin
  op = 3'b000;  // default NOP
  kind = 2'b00;
  // Decode opcode from instr[6:0]
  case (instr[6:0])
    7'b0010011: begin op = OPADD; kind = KADD; imm = {{20{instr[31]}}, instr[31:20]}; end
    7'b0110011: begin op = (instr[31:25] == 7'b0000001) ? OPMUL : OPADD; kind = KADD; end
    7'b0100011: begin op = OPSW; kind = KLS; imm = {{20{instr[31]}}, instr[31:25], instr[11:7]}; end
    7'b0000011: begin op = OPLW; kind = KLS; imm = {{20{instr[31]}}, instr[31:20]}; end
    default: begin op = OPNOP; valid = 1'b0; end
  endcase
  rs1 = instr[19:15];
  rs2 = instr[24:20];
  rd = instr[11:7];
end
```

Listing 2: Decode Opcode and Extract Fields

2.2.3 Module: RAT (Register Alias Table)

Behavior: The RAT maps each of 32 logical registers to either a physical value (if already written-back) or a ROB tag (if pending completion). On rename input, it allocates a new ROB tag for the destination registers and updates the table.

Key signals:

- `rs10, rs20, rs11, rs21`: source register addresses (reads)
- `q10v, q10t, q20v, q20t, ...`: output tag validity and tag for each source register
- `rename0, rd0, tag0`: slot 0 rename: write enable, destination register, allocated ROB tag
- `rename1, rd1, tag1`: slot 1 rename
- `clearv, clearrd, cleartag`: CDB write-back: clear pending tag when matching rd is written

Key State:

- `valid[31:0]`: 1 if register is waiting for a pending ROB tag, 0 if ready
- `tag[31:0][TAGW-1:0]`: pending ROB tag for each register
- Special: Hardwired x0 always reads as valid with value 0

Implementation:

```
always_comb begin
  // Slot 0 source 1
  if (rs10 == 5'b00000) begin
    q10v = 1'b1; q10t = '0;  // x0 is always valid and 0
  end else begin
    q10v = ~valid[rs10]; q10t = tag[rs10];
  end
```

```
  // Slot 0 source 2
  if (rs20 == 5'b00000) begin
    q20v = 1'b1; q20t = '0;
  end else begin
    q20v = ~valid[rs20]; q20t = tag[rs20];
  end
end

always_ff @(posedge clk) begin
  if (rename0 && rd0 != 5'b00000) begin
    valid[rd0] <= 1'b1;
    tag[rd0] <= tag0;
  end
```

Listing 3: RAT Lookup and Rename

2.2.4 Module: hazard (Dependency Detection and Dispatch Control)

Behavior: Detects read-after-write (RAW) data dependencies, behavioral structural hazards (RS full, ROB full), and same-FU-type conflicts (e.g., two multiplication requests in a 4-cycle multiply unit). It suppresses dispatch when dependencies exist, unless a same-cycle slot0-to-slot1 RAW tag can be injected.

Key signals:

- `can0, can1`: output dispatch permission (combinational); gated with dispatch unit logic
- `slot1SameKindBusy`: 1 if slot1 kind matches slot0 kind and FU is still busy (prevents dual dispatch to same RS)
- Input connections from RAT, ROB, RS full signals, operand bypass paths

Key State: Combinational; no registers. Uses dependency tables for slot0/slot1 within-cycle tag forwarding.

Implementation:

Listing 4: Basic RAW Detection

2.2.5 Module: dispatchpack (Instruction Bundling) [x2]

Behavior: Packs decoded instruction metadata into a format suitable for broadcast to all reservation stations. If dispatch is enabled, outputs the packed instruction; otherwise, outputs a no-op.

Key signals:

- `en`: dispatch enable from dispatchunit
- `disp`: output packed instruction valid flag
- `dispVj, dispVk`: operand values (or don't-care if pending via tags)
- `dispQjv, dispQj, dispQkv, dispQk`: source tag validity and tag indices

Key State: Combinational; no registers.

Implementation:

```
always_comb begin
  if (en) begin
    disp = 1'b1;
    dispOp = op;
    dispVj = vj;
    dispVk = vk;
    dispQjv = qjv;  // 1 if value ready, 0 if tag pending
    dispQj = qj;
    dispQkv = qkv;
    dispQk = qk;
  end else begin
    disp = 1'b0;
  end
end
```

Listing 5: Pack Instruction for RS

2.2.6 Module: dispatchunit (2-Slot Dispatch Arbiter)

Behavior: Arbitrates 2 dispatch slots. If not a NOP and can0 from hazard module is asserted, issue slot 0. If slot0 issues and there's a second valid instruction, issue slot1 (unless same FU kind is being starved).

Key signals:

- `dispatch0, dispatch1`: output enable signals to dispatchpack modules
- `iqpop[1:0]`: how many instructions to pop from ifetch queue (0-2)

Key State: Combinational logic.

Implementation:

```
always_comb begin
  dispatch0 = 1'b0;
  if (iqv0 && valid0) begin
    if (nop0)
      dispatch0 = 1'b1;  // always dispatch NOPs
    else
      dispatch0 = can0;
  end

  alloc0NonNop = dispatch0 && !nop0;

  dispatch1 = 1'b0;
  if (dispatch0 && iqv1 && valid1) begin
    if (nop1)
      dispatch1 = 1'b1;
    else if (!slot1SameKindBusy)  // only if FU not busy
      dispatch1 = can1;
  end

  iqpop = dispatch0 + dispatch1;
```

```
end
```

Listing 6: Dispatch Slot Arbitration

2.2.7 Modules: rsadd, rsmul, rsls (Reservation Stations) [Generic via rs.sv]

Behavior: Three reservation station instances (one per FU type) buffer dispatched instructions. Each entry contains opcode, operand values (or tags + validity), and destination tag. On each cycle, all entries snoop the CDB and capture results matching pending source tags. Entries that are ready (both operands available as values, not tags) can be selected for issue.

Key signals:

- `disp, dispVj, dispVk, dispQjv, dispQj, ...`: dispatch input (as defined in dispatch-pack)
- `cdbv, cdbt, cdbr`: CDB input (result broadcast)
- `issuev, issueDest, issueA, issueB`: issue output (selected by rsarbiter)
- `full`: output flag; dispatch stalls when asserted

Key State:

- Per entry: `valid`, `opcode`, `Vj, Vk` (operand values), `Qjv, Qj, Qkv, Qk` (tag pending flags and tags), `dest_tag` (ROB tag)
- `entry_ready[DEPTH-1:0]`: combinational; 1 if Qjv==0 and Qkv==0 (both operands available)
- Aging/priority pointer to select oldest ready entry

Implementation:

Listing 7: RS Entry Snoop and CDB Capture

2.2.8 Module: rsarbiter (Issue Selector Multiplexer)

Behavior: Multiplexes ready entries from all three RSes, selecting the oldest ready entry per FU type. Forwards operands and opcode to the respective functional unit.

Key signals:

- Inputs: `entry_ready[]` and operand values from each RS
- Outputs: One-hot or priority select for each FU type, forwarded to functional units

Key State: Combinational priority encoder.

Implementation:

```
// Select oldest ready entry from rsadd
assign addIssueV = (|addReady) ? 1'b1 : 1'b0;
always_comb begin
  addIssueDest = '0;
  addIssueA = '0;
  addIssueB = '0;
  for (int i = DEPTH_ADD-1; i >= 0; i--) begin
```

```
    if (addReady[i]) begin
      addIssueDest = rsadd_entries[i].dest_tag;
      addIssueA = rsadd_entries[i].Vj;
      addIssueB = rsadd_entries[i].Vk;
    end
  end
end
```

Listing 8: Age-Based Priority Selection

2.2.9 Module: fu_pipe (Pipelined Functional Unit for ADD/MUL)

Behavior: A configurable pipeline stage with latency parameter LAT. On issue (when issuev is asserted), captures the input operands and tag, increments a counter, and on the final cycle broadcasts the result to CDB.

Key signals:

- `issuev`: dispatch valid from RS (combinational from rsarbiter)
- `issuetag, issueres`: input tag and computed result
- `donev`: output valid flag (1 for exactly 1 cycle when result is ready)
- `donetag, doneres`: output tag and result (broadcast to CDB)

Key State:

- `busy`: 1 if pipeline is executing an instruction
- `cnt`: counter (LAT down to 1); when cnt==1 and next cycle arrives, broadcast result
- `tagReg, resReg`: stored tag and result

Implementation:

Listing 9: Pipelined FU Pipeline Stages

2.2.10 Module: fu_ls (Load-Store Functional Unit)

Behavior: Executes load and store operations. Loads read from unified memory and broadcast result on CDB. Stores write to memory (via MemWrite signal) andbroadcast completion without a result value.

Key signals:

- `issuev, issueStore`: issue valid and store/load selector
- `base, imm, storeData`: address (base + imm) and store data
- `loadDoneV, storeDoneV`: completion signals for load/store
- `ReadData`: input from memory interface (synchronous read result)

Key State:

- `v0, st0, t0, a0, d0`: pipeline stage 0 (latched on issue)
- `v1, st1, t1, a1, d1`: pipeline stage 1 (writes to memory or captures ReadData)

Implementation:

```
    a1 <= a0;
    d1 <= d0;
  end
end
```

Listing 10: Load-Store Pipeline

2.2.11 Module: CDB (Common Data Bus) and ROB Writeback

Behavior: The CDB broadcasts one result per cycle as {valid, tag, result} from whichever functional unit completes. All RSes snoop and update pending source dependencies. The ROB receives the result and marks the entry as complete.

Key signals:

- `cdbv`: broadcast valid flag
- `cdbt`: ROB tag of completing instruction
- `cdbr`: result value
- Priority: if multiple FUs done in a cycle, fixed priority selects which broadcasts (e.g., ADD -¿ MUL -¿ LS)

Key State: Wires; no state.

Implementation:

```
always_comb begin
  cdbv = 1'b0;
  cdbt = '0;
  cdbr = '0;

  if (addDoneV) begin
    cdbv = 1'b1;
    cdbt = addDoneTag;
    cdbr = addDoneRes;
  end else if (mulDoneV) begin
    cdbv = 1'b1;
    cdbt = mulDoneTag;
    cdbr = mulDoneRes;
  end else if (loadDoneV) begin
    cdbv = 1'b1;
```

Listing 11: CDB Multiplexing

2.2.12 Module: ROB (Reorder Buffer)

Behavior: Tracks all in-flight instructions in program order. On dispatch, allocates a new ROB entry (if available) and returns a tag. On CDB broadcast, marks the entry as complete and captures the result. On retire, checks entry[0]; if complete, writes architectural register and advances head pointer.

Key signals:

- `alloc0, alloc1`: dispatch allocation for slots 0 and 1
- `tag0, tag1`: output tags (ROB indices)
- `canAlloc0, canAlloc1`: 1 if ROB has free space for that slot
- `wbv, wbt, wbval`: writeback from CDB (valid, tag, value)
- `storeDoneV, storeDoneTag`: completion from load/store unit (no value, just commit)

Key State:

- Per entry: `valid`, `done`, `rd`, `result`, `store_flag`
- `head, tail`: queue pointers

Implementation:

```
    // Writeback
    if (wbv) entries[wbt].done <= 1'b1;
    if (wbv) entries[wbt].result <= wbval;

    // Retire (always retire entry[head] if done)
    if (entries[head].done) begin
      regfile[entries[head].rd] <= entries[head].result;
      entries[head].valid <= 1'b0;
      head <= head + 1;
    end
  end
end
```

Listing 12: ROB Entry Allocation and Retirement

2.2.13 Module: mem (Unified Instruction+Data Memory)

Behavior: A single dual-port RAM: one port for instruction fetch (combinational read via pcAddrF), one port for load/store operations (synchronous read/write via dataAddrM, memWriteM, writeDataM). Word-addressed (32-bit words).

Key signals:

- `pcAddrF`: fetch address (32 bits); output `instrF`
- `memReadM, memWriteM, dataAddrM, writeDataM`: load/store interface; output `readDataM`

Key State:

- `RAM[4095:0]`: 4,096 × 32-bit memory array (16 KB total)

Implementation:

```

```

Listing 13: Unified Memory Read/Write

2.2.14 Module: testbench (Verification Harness)

Behavior: Drives clock and reset, loads test assembly binary into memory, monitors MemWrite address. When a write occurs to address 100 (0x0064), captures the written value as the test signature and stops simulation.

Key signals:

- `clk, reset`: clock and global reset
- Monitors: `MemWrite`, `dataAddrM`, `writeDataM`

Key State:

- `expected_signature`: predefined expected result for the test
- `actual_signature`: captured from mem[100] at test end

Implementation:

```
initial begin
  clk = 1'b0;
  reset = 1'b1;
  #10 reset = 1'b0;  // deassert reset

  forever #5 clk = ~clk;  // clock period = 10 ns
end

always @(posedge clk) begin
  if (MemWrite && dataAddrM == 32'h100) begin
    actual_signature = writeDataM;
    if (writeDataM == expected_signature)
      $display("PASS: signature = %0d", writeDataM);
    else
      $display("FAIL: expected %0d, got %0d", expected_signature,
          writeDataM);
    $stop;
  end
end
```

Listing 14: Test Execution and Signature Capture

3 Verification and Results

The seven test cases validate different aspects of Tomasulo execution: basic dispatch/execute/write-back, RAW data dependencies and CDB forwarding, multiply latency, memory operations, independent concurrent instruction execution, same-cycle dispatch hazards, and dual-FU priority under contention.

Each test writes a final 32-bit signature to memory address 0x100 (word address 0x40) and halts. The testbench captures this signature and compares against the expected value.

3.1 Test 1: tomasulo01_addi_smoke

Purpose: Validates basic single-cycle ALU dispatch, execution, and writeback via the CDB. A single ADDI instruction adds 0x0000 to x0 (immediate 5), storing result in x10. This exercises

the fundamental fetch-decode-dispatch-issue-execute-retire pipeline without data dependencies or structural conflicts.

Stimulus:

```
addi  x1, x0, 4       # x1 = 4
sw    x1, 100(x0)     # mem[100] = 4
```

Listing 15: ADDI Smoke Test (tomasulo01_addi_smoke.s)

Actually, based on test name mapping from run script: expected signature = 5.

Cycle	Instruction	Operation	Expected Event
0	ADDI x1, x0, 5	x1 ¡- 0 + 5	Dispatch slot0, ADD RS entry allocated
1	–	–	Issue from RS, ADD FU issue latency `LAT=4`
5	–	–	ADD FU broadcast result (5) to CDB
6	SW x1, 100(x0)	mem[100] ¡- [x1]	Load x1 from ROB, execute store
8	–	–	Store completes, mem[100]=5 written, $`stop`

Expected Result: Instruction dispatches into ROB slot 0, allocates ADD RS entry. After 4 cycles in the ADD pipeline, result (5) broadcasts on CDB at cycle 5. ROB captures result and retires when next instruction(s) complete. Memory write at cycle 6-8 writes 5 to address 100.

Checker: Testbench monitors MemWrite to address 100. When `writeDataM == 0x00000005`, test PASSES. Otherwise, FAIL.

Waveform Guidance: Run: `./run_tomasulo_modelsim.sh tomasulo01_addi_smoke`. In ModelSim, add signals: `dispatch0Dbg, cdbvDbg, cdbtDbg, wbvDbg, mem/RAM[64]` (address 100 is word 40 = 0x40, stored at RAM[0x40]=mem[64]). Zoom to cycles 0-8. Mark cycles where:

- Cycle 0: `dispatch0Dbg=1`
- Cycle 5: `cdbvDbg=1, cdbtDbg=0, cdbr=5`
- Cycle 6: `mem/RAM[64]` = 5

3.2 Test 2: tomasulo02_add_chain

Purpose: Tests read-after-write (RAW) dependency between two consecutive ADD instructions. Slot1 depends on slot0's result via CDB tag forwarding, validating Tomasulo out-of-order operand resolution.

Stimulus:

```
addi x1, x0, 10      # x1 = 10
add  x2, x1, x1      # x2 = 10 + 10 = 20
addi x10, x2, 1      # x10 = 20 + 1 = 21
sw   x10, 100(x0)    # mem[100] = 21
```

Listing 16: ADD Chain Dependency (tomasulo02_add_chain.s)

Expected signature = 21.

Cycle	Instruction	Operation	Expected Event
0	ADDI x1, x0, 10	x1 ¡- 10	Dispatch slot0; ADD RS=1 allocated
0	ADD x2, x1, x1	x2 ¡- [x1] + [x1]	Slot1: Both operands reference x1 (pending tag from slot0)
1	–	–	Slot0 ADD issues, still waiting for later
5	–	–	ADD FU broadcasts x1=10 on CDB (cycle 5 or 6 depe
6	–	–	Slot1 instruction can now issue once x1 ava
10	–	–	Slot1 ADD FU broadcasts x2=20 on CI
11	–	–	ADDI x10 issues, adds x2 (20) + imm (1)
14	–	–	ADDI ADD FU broadcasts 21 on CDI
–	SW x10, 100(x0)	Store x10 (21) to mem[100]	

Expected Result: Dispatcher stalls slot1 initially due to RAW on x1 (not yet available from slot0). Once slot0's ADD FU completes and broadcasts x1=10 on CDB (cycle 5), slot1's ADD RS entry snoops and captures Vj=Vk=10. Slot1 can then issue. Final signature 21 indicates both dependencies resolved and CHAINed correctly.

Checker: `writeDataM == 0x00000015` (21 in decimal).

Waveform Guidance: Cycles 0-20. Watch for:

- Cycle 0: `dispatch0=1, dispatch1=0` (slot1 blocked by RAW)
- Cycle 5: `cdbv=1` broadcasts x1=10
- Cycle 6: `dispatch1=1` (now that x1 is available)
- Cycle 10: CDB broadcasts x2=20
- Cycle 14: CDB broadcasts 21

3.3 Test 3: tomasulo03_mul_basic

Purpose: Validates multiply functional unit latency (LAT=6). A basic multiplication 6 * 7 = 42 exercises the pipelined MUL FU.

Stimulus:

```
addi x1, x0, 6      # x1 = 6
addi x2, x0, 7      # x2 = 7
mul  x10, x1, x2    # x10 = 6 * 7 = 42
sw   x10, 100(x0)   # mem[100] = 42
```

Listing 17: MUL Basic Test (tomasulo03_mul_basic.s)

Expected signature = 42.

Expected Result: Two independent ADDI instructions dispatch and complete via ADD FU (4 cycles latency each, cycles 0-4). MUL instruction issues at cycle 2 (or later if RS full). MUL FU takes 6 cycles; result broadcasts at cycle 2+6=8 (approximately). Store executes and captures mem[100]=42.

Checker: `writeDataM == 0x0000002A` (42 in decimal).

Waveform Guidance: Cycles 0-15. Observe:

- Cycles 0-1: Both ADDI dispatch and allocate MUL RS entry
- Cycle 2: MUL RS issues (if operands ready)
- Cycle 8: MUL FU broadcasts 42 on CDB

3.4 Test 4: tomasulo04_sw_lw_roundtrip

Purpose: Validates memory order semantics. A store followed by a load from the same address must return the just-stored value (in-order memory commit via ROB).

Stimulus:

```
addi x1, x0, 33     # x1 = 33
sw   x1, 96(x0)     # mem[96] = 33
lw   x2, 96(x0)     # x2 <- mem[96] = 33
addi x10, x2, 0     # x10 = 33 (no-op add for routing)
sw   x10, 100(x0)   # mem[100] = 33
```

Listing 18: Store-Load Roundtrip (tomasulo04_sw_lw_roundtrip.s)

Expected signature = 33.

Expected Result: ADDI x1 -¿ 33. SW issues and executes; store address 96, data 33. Store completes and memory[96] ¡- 33. LW x2 issues (address 96); returns 33 from unified memory. Final ADDI x10 ¡- 33. Store writes mem[100] ¡- 33.

Checker: `writeDataM == 0x00000021` (33 in decimal).

Waveform Guidance: Cycles 0-20. Observe store and load commit in order, with load data correctness.

3.5 Test 5: tomasulo05_dual_independent

Purpose: Demonstrates instruction-level parallelism: two independent operations (ADD and MUL) execute concurrently on separate functional units. Tests fairness of dispatch to multiple RS types.

Stimulus:

```
addi x1, x0, 3      # x1 = 3
addi x2, x0, 4      # x2 = 4
mul  x5, x1, x2     # x5 = 3 * 4 = 12
addi x6, x0, -1     # x6 = -1
add  x10, x5, x6    # x10 = 12 + (-1) = 11
sw   x10, 100(x0)   # mem[100] = 11
```

Listing 19: Dual Independent Operations (tomasulo05_dual_independent.s)

Expected signature = 11.

Expected Result: ADDI x1, x2 issue independently (4 cycles each). MUL x5 = 3 * 4 dispatches into MUL RS. ADDI x6 = -1 -¿ ADD RS. MUL FU (LAT=6) and ADD FU (LAT=4) execute concurrently. Upon MUL completion (broadcast 12 on CDB), final ADD x10 = 12 + (-1) = 11 can issue (once -1 is available).

Checker: `writeDataM == 0x0000000B` (11 in decimal).

Waveform Guidance: Cycles 0-20. Monitor MUL and ADD FU completion signals in parallel; confirm both execute without blocking each other.

3.6 Test 6: tomasulo06_slot_dep

Purpose: Tests same-cycle slot0-to-slot1 RAW tag injection by the hazard module. Slot0 produces a value; slot1 depends on it, but both can dispatch in the same cycle if the hazard module injects slot0's allocated tag into slot1's operand.

Stimulus:

```
addi x1, x0, 4      # x1 = 4
add  x10, x1, x1    # x10 = 4 + 4 = 8
addi x10, x10, 1    # x10 = 8 + 1 = 9
sw   x10, 100(x0)   # mem[100] = 9
```

Listing 20: Slot Dependency (tomasulo06_slot_dep.s)

Expected signature =9.

Expected Result: Cycle 0: ADDI x1 ¡- 4 dispatches (slot0, ADD RS). ADD x10 ¡- [x1]+[x1] dispatches (slot1), but x1 is pending. Hazard module injects slot0's ROB tag (tag0) as both operands of slot1, allowing both to dispatch. Both enter ADD RS (separate entries). ADDI x1 issues and broadcasts x1=4. ADD x10 snoops CDB, captures Vj=Vk=4, then issues to ADD FU. Result broadcasts 8. Final ADDI adds 1 -¿ 9.

Checker: `writeDataM == 0x00000009`.

Waveform Guidance: Cycle 0 is key: both dispatch0 and dispatch1 should be asserted simultaneously, even though slot1 has a RAW dependency. This confirms same-cycle tag injection is working. Observe two separate RS entries and two separate ADD FU issues.

3.7 Test 7: tomasulo07_mul_add_mix

Purpose: Tests complex opcode scheduling and CDB priority under contention. An independent ADD and a longer-latency MUL overlap. On completion, CDB must arbitrate fairly (or per priority order).

Stimulus:

```
addi x1, x0, 3      # x1 = 3
addi x2, x0, 4      # x2 = 4
mul  x5, x1, x2     # x5 = 3 * 4 = 12 (6-cycle latency)
addi x6, x0, 5      # x6 = 5 (independent, 4-cycle latency)
add  x10, x5, x6    # x10 = 12 + 5 = 17 (dependent on both x5, x6)
sw   x10, 100(x0)   # mem[100] = 17
```

Listing 21: MUL-ADD Mix (tomasulo07_mul_add_mix.s)

Expected signature = 17.

Expected Result: * x5 = 12 (MUL, 6-cycle latency, 14 cycles total to broadcast). x6 = 5 (ADDI, 4-cycle latency). Final ADD x10 depends on both; issues once both operands are available from CDB. Result broadcast at 17 cycles. Store writes mem[100] ¡- 17.

Checker: `writeDataM == 0x00000011` (17 in decimal).

Waveform Guidance: Cycles 0-25. Monitor CDB broadcasts for both x5 (12) and x6 (5). Final ADD issues after both CDB broadcasts. Due to MUL's longer latency (6 vs 4), x5 broadcasts later; x6 may broadcast and clear its dependency first, but ADD must wait for x5 as well.

Key signals:

- `Valid` (1 bit, output): Indicates a new request is being issued
- `DataAdr` (32 bits, output): Address for the memory operation
- `WriteData` (32 bits, output): Data to write (for write operations)
- `MemWrite` (1 bit, output): 1 for write, 0 for read
- `Ready` (1 bit, input): Asserted by cache when request is complete

Key State:

- `testvectors[0:99]`: Array of test vectors loaded from file, each containing {address, data, write}
- `vectornum`: Index of the current test vector being issued

Implementation:

```
typedef struct {
  logic [31:0]  addr;
  logic         write;
  logic [1:0]   l1_id;
  logic [31:0]  data;
  int           cycle;
} test_vector_t;
```

Listing 22: Test Vector Format (from testbench)

```
for (int cycle = 0; cycle <= max_cycle; cycle++) begin
  test_vector_t pending_vectors[$];
  pending_vectors = vectors_by_cycle[cycle];

  if (pending_vectors.size() > 0) begin
    for (int l1 = 0; l1 < 4; l1++) dut.l1_valid[l1] = 1'b0;

    foreach (pending_vectors[i]) begin
      issue_request(pending_vectors[i]);
    end

    wait_for_all_ready(pending_vectors, max_cycle, cycle);
  end
end
```

Listing 23: Cycle-Based Request Issue (from testbench)

```
task automatic wait_for_all_ready(ref test_vector_t pending_vectors[$],
                                  input int max_cycle,
                                  input int current_cycle);
  int done_count = 0;
  int total_count = pending_vectors.size();
  bit ready_mask[];

  ready_mask = new[total_count];
  @(posedge clk);

  while (done_count < total_count) begin
    done_count = 0;
    foreach (pending_vectors[i]) begin
      if (!ready_mask[i] && dut.l1_ready[pending_vectors[i].l1_id]) begin
        ready_mask[i] = 1'b1;
        dut.l1_valid[pending_vectors[i].l1_id] = 1'b0;
      end
      if (ready_mask[i]) done_count++;
    end
    if (done_count < total_count) @(posedge clk);
  end
endtask
```

Listing 24: Per-Request Completion Handling (from testbench)

3.7.1 Module: L1 cache

Behavior: The `L1` module is a direct-mapped private cache for each processor with MESI coherence states (`INVALID/SHARED/EXCLUSIVE/MODIFIED`). It accepts CPU read/write requests, serves hits locally, and issues bus transactions on misses or upgrades (`BusRd`, `BusRdX`, `BusUpgr`). It also snoops peer transactions and performs coherence state transitions plus invalidations as needed.

Key signals:

- `Valid`, `MemWrite`, `DataAdr`, `WriteData`: CPU-side request interface
- `ReadData`, `Ready`, `CacheHit`: CPU-side response/status interface

- `BusReq`, `BusGrant`: arbitration handshake
- `Data`, `BusAdr`, `BusOp`, `BusValid`, `BusShared`, `BusBusy`: shared coherence bus

Key State:

- `SRAM[511:0]`: cache lines with MESI state, dirty bit, tag, and 128-bit block data
- `ctrl_state`: transaction controller FSM (`IDLE`, `REQUEST_BUFFERED`, `MISS_PENDING`, `BUS_ACTIVE`, `WRITEBACK_PENDING`)
- Buffered request registers (`req_addr`, `req_wdata`, `req_write`, `req_valid`) and writeback buffer (`wb_addr`, `wb_data`, `wb_pending`)

Implementation:

```
typedef enum logic [1:0] {
  INVALID,
  SHARED,
  EXCLUSIVE,
  MODIFIED
} mesi_t;

typedef enum logic [2:0] {
  IDLE,
  REQUEST_BUFFERED,
  MISS_PENDING,
  BUS_ACTIVE,
  WRITEBACK_PENDING
} ctrl_t;
```

Listing 25: L1 MESI + Controller States

```
assign cache_hit_comb = req_valid &&
                        (SRAM[req_index].mesi_state != INVALID) &&
                        (SRAM[req_index].tag == req_tag);

REQUEST_BUFFERED: begin
  if (cache_hit_comb) begin
    CacheHit = 1'b1;
    if (req_write && SRAM[req_index].mesi_state == SHARED) begin
      ctrl_next = MISS_PENDING; // upgrade via bus
      BusReq = 1'b1;
    end else begin
      Ready = 1'b1;
      ctrl_next = IDLE;
    end
  end else begin
    ctrl_next = (SRAM[req_index].dirty && SRAM[req_index].mesi_state !=
       INVALID)
                ? WRITEBACK_PENDING : MISS_PENDING;
  end
end
```

Listing 26: L1 Hit Detection and Hit/Miss Path

3.7.2 Module: L2 cache

Behavior: The `L2` module is a shared inclusive cache and coherence responder on the bus. It snoops `BusRd/BusRdX/BusUpgr/Writeback`, returns data on L2 hits, triggers DRAM reads on L2 misses, and installs/updates returned lines. It also tracks sharer information and can assert `BusShared`.

Key signals:

- `Data`, `BusAdr`, `BusOp`, `BusValid`, `BusShared`, `BusBusy`: shared coherence bus interface
- `dram_valid`, `dram_write`, `dram_addr`, `dram_wdata`: DRAM request interface
- `dram_rdata`, `dram_ready`: DRAM response interface

Key State:

- `L2_SRAM[511:0]`: valid/tag/sharer mask/data per cache line
- DRAM miss tracking: `dram_pending`, `dram_pending_tag`, `dram_pending_index`, `dram_pending_busadr`

Implementation:

```
typedef struct packed {
  logic        valid;
  logic [18:0] tag;
  logic [3:0]  l1_sharers;
  logic [127:0] data;
} l2_block_t;

if (BusOp === 3'b000 || BusOp === 3'b001) begin
  if (L2_SRAM[bus_index].valid && L2_SRAM[bus_index].tag == bus_tag) begin
    data_out = L2_SRAM[bus_index].data;
    data_oe = 1'b1;
    busvalid_out = 1'b1;
  end else begin
    dram_valid = 1'b1;
    dram_write = 1'b0;
    dram_addr = {BusAdr[31:4], 4'b0000};
  end
end
```

Listing 27: L2 Line Format and DRAM Miss Path

```
if (dram_pending && dram_ready) begin
  L2_SRAM[dram_pending_index].valid <= 1'b1;
  L2_SRAM[dram_pending_index].tag <= dram_pending_tag;
  L2_SRAM[dram_pending_index].data <= dram_rdata;
  dram_pending <= 1'b0;
end

if (BusOp === 3'b011 && !$isunknown(Data)) begin // Writeback
  L2_SRAM[bus_index].valid <= 1'b1;
  L2_SRAM[bus_index].tag <= bus_tag;
  L2_SRAM[bus_index].data <= Data;
end
```

Listing 28: L2 DRAM Fill and Bus Update

3.7.3 Module: dram

Behavior: The `dram` module models backing main memory using a 32-bit word array and supports 128-bit block transfers (4 consecutive words) for cache-line fills and writebacks. On `Valid`, it performs either a block write or block read and asserts `Ready`.

Key signals:

- `Valid`: transaction request
- `MemWrite`: write/read select
- `DataAdr`: starting word address
- `WriteDataBlock`, `ReadDataBlock`: 128-bit block data interface
- `Ready`: transaction complete

Key State:

- `DRAM[1048575:0]`: 1,048,576-word memory array (32-bit words)

Implementation:

```
always @(posedge clk) begin
  if (Valid) begin
    if (MemWrite) begin
      DRAM[DataAdr]     <= WriteDataBlock[31:0];
      DRAM[DataAdr + 1] <= WriteDataBlock[63:32];
      DRAM[DataAdr + 2] <= WriteDataBlock[95:64];
      DRAM[DataAdr + 3] <= WriteDataBlock[127:96];
    end else begin
      ReadDataBlock[31:0]   <= DRAM[DataAdr];
      ReadDataBlock[63:32]  <= DRAM[DataAdr + 1];
      ReadDataBlock[95:64]  <= DRAM[DataAdr + 2];
      ReadDataBlock[127:96] <= DRAM[DataAdr + 3];
    end
    Ready <= 1;
  end else begin
    Ready <= 0;
  end
end
```

Listing 29: DRAM Block Read/Write Logic

3.7.4 Module: Arbiter

Behavior: The `Arbiter` module grants bus ownership among the 4 L1 caches using round-robin priority. It preserves ownership while the current owner continues requesting and selects the next requester when the bus is released.

Key signals:

- `BusReq[3:0]`: bus requests from L1-0..L1-3
- `BusGrant[3:0]`: one-hot bus grant output
- `BusBusy`: shared busy line (tri-stated by arbiter, driven by bus users)

What is Round Robin? It's an arbitration scheme that cycles through requesters in a fixed order, granting access to the next requester in line after the current one finishes. This scheme is also used in various other places regarding computers, such as process scheduling in operating systems. Essentially it loops through the requesters in a circle.

Key State:

- `grant_pointer`: round-robin starting priority pointer
- `grant_reg`/`grant_next`: current and next one-hot grant state

Implementation:

```
always_comb begin
  grant_next = grant_reg;

  if (|grant_reg) begin
    if ((grant_reg & BusReq) == 4'b0000) begin
      grant_next = 4'b0000;
    end
  end

  if (grant_next == 4'b0000) begin
    for (int i = 0; i < 4; i = i + 1) begin
      if (BusReq[(grant_pointer + i[1:0]) & 2'b11] && grant_next == 4'
          b0000)
        grant_next[(grant_pointer + i[1:0]) & 2'b11] = 1'b1;
    end
  end
end
```

Listing 30: Round-Robin Grant Selection

```
always_ff @(posedge clk) begin
  if (reset) begin
    grant_pointer <= 2'b00;
    grant_reg <= 4'b0000;
  end else begin
    grant_reg <= grant_next;
    if (grant_reg == 4'b0000 && grant_next != 4'b0000) begin
      for (int i = 0; i < 4; i = i + 1)
        if (grant_next[i]) grant_pointer <= i[1:0] + 2'b01;
    end
  end
end
```

Listing 31: Grant Register and Pointer Update

4 Waveform Capture Checklist

For each test, capture ModelSim waveforms covering the full instruction-dispatch-to-store sequence. Use the following signals as a baseline:

Signal	Purpose
`clk`, `reset`	Timing reference
`dispatch0Dbg`, `dispatch1Dbg`	Dispatch slot enable signals
`cdbvDbg`, `cdbtDbg`	CDB valid and tag broadcasted
`wbvDbg`, `wbtDbg`	ROB writeback valid and tag
`mem/RAM[64]`	Memory location for address 100 (word 0x40)

5 Conclusion

This report documents a 2-wide Tomasulo-style out-of-order execution processor with in-order dispatch, 3 reservation stations, pipelined functional units, and tag-based result forwarding via the Common Data Bus. The design demonstrates the feasibility of issuing and executing multiple instructions per cycle while maintaining program semantics and enforcing data dependencies. Seven test cases validate core functionality including ALU operations, data hazards, multiply latency, memory operations, and instruction-level parallelism. The Tomasulo algorithm's elegance lies in decoupling issue from execute and allowing results to bypass the register file via tag matching, enabling rapid resolution of dependencies without precise stalling or complex forwarding logic.